

university of groningen

Performance Engineering: Tutorial #01

WMCS045-05

Dirk Pleiter

Bernoulli Institute

2026-09-03

Overview



1. Time Measurements
2. gprof
3. PAPI
4. perf
5. Score-P / Scalasca
6. HPCToolkit
7. Final

Content



1. **Time Measurements**
2. gprof
3. PAPI
4. perf
5. Score-P / Scalasca
6. HPCToolkit
7. Final

Time Tool



- `/usr/bin/time`
  - Shows **elapsed time** + **CPU utilization** by user and system
  - Resolution given by ticks per second: `sysconf(_SC_CLK_TCK)`
  - Typical resolution: $10\,\text{ms}$

Tool `clock_gettime()`



- Function defined in the **Portable Operating System Interface (POSIX) Base Specifications** 1003.1 [IEEE; 2024]
  - Proprietary additions for specific OSs, e.g. Linux
- **Function declaration**

```c
#include <time.h>
int clock_gettime(clockid_t clock_id, struct timespec *tp);
```

- **Function specification**: The function shall return the current value (`tp`) for the specified clock (`clock_id`)
- Return value definition

```c
struct timespec {
    time_t tv_sec;  /* seconds */
    long tv_nsec;   /* nanoseconds */
};
```

Tool `clock_gettime()`: Clocks



- Selected choice of clocks
  - `CLOCK_MONOTONIC` (POSIX)
    - Monotonically incremented clock
    - The return value represents the amount of time since an unspecified point in the past
      - Linux: Time since boot of the system (possibly excluding time when system was suspended)
    - It is not possible to set this clock
  - `CLOCK_THREAD_CPUTIME_ID` (Linux)
    - The clock measures the CPU core utilisation of the thread that called the function
  - `CLOCK_PROCESS_CPUTIME_ID` (Linux)
    - The clock measures the CPU core utilisation of all threads of a process

Timer Resolution



- The resolution of the used timer must be small compared to the latency that one tries to measure
- **Function specification**: [IEEE; 2024]
  - The function shall return the resolution of any clock
  - Clock resolutions are implementation-defined and cannot be set by a process
- **Function declaration**

```c
#include <time.h>
int clock_getres(clockid_t clock_id, struct timespec *res);
```

Tool `clock_gettime()`: Example



```c
#include <stdio.h>
#include <time.h>

int main()
{
    struct timespec t0, t1;

    clock_gettime(CLOCK_BOOTTIME, &t0);
    mykernel();
    clock_gettime(CLOCK_BOOTTIME, &t1);

    double t = (t1.tv_sec  - t0.tv_sec) +
               (t1.tv_nsec - t0.tv_nsec) * 1e-9
    printf("t=%e\n", t);
}
```

Clock-Cycle Counters: x86 (1/2)



- x86 assembler instructions `RDTSC` and `RDTSD` allow to obtain the **Time Stamp Counter (TSC)**
  - TSC counts the number of clock cycles
  - TSC frequency depends on the CPU model but is typically **close to the nominal CPU frequency**
  - The instructions are not available on all processors
    See `/proc/cpuinfo`
  - An unsigned 64-bit counter is returned
    ☞ No need to worry about counter overflow
- **Limitations**
  - Overhead of instructions to capture the clock
  - Potential out-of-order execution issues
  - Conversion to real time: core clock may change
    (only on some model affects time-stamp counter)

Clock-Cycle Counters: x86 (2/2)



Example code:

```
1 #include <stdint.h>
2
3 inline uint64_t get_rdtsc(void)
4 {
5   uint32_t lo, hi;
6   __asm__ __volatile__ (       // serialize
7     "xorl %%eax,%%eax \n        cpuid"
8     ::: "%rax", "%rbx", "%rcx", "%rdx");
9     /* We cannot use "=A", since this would use %rax on x86_64 and
10        return only the lower 32bits of the TSC */
11  __asm__ __volatile__ ("rdtsc" : "=a" (lo), "=d" (hi));
12  return ((uint64_t) hi << 32 | lo);
13 }
```

Clock-Cycle Counters: Arm



- **Generic Timer**: Standardized timer framework for Arm cores
- Different registers, e.g. `CNTVCT` (Counter-timer Virtual Count)
- The counter is incremented at a rate that corresponds to a 1 ns **resolution**
- **Example code**:

```
1 #include <inttypes.h>
2
3 inline uint64_t get_armgentimer(void)
4 {
5   uint64_t cc = 0;
6
7   asm volatile("mrs %0,  CNTVCT_EL0" : "=r" (cc));
8
9   return cc;
10 }
```

Content



1. Time Measurements
2. gprof
3. PAPI
4. perf
5. Score-P / Scalasca
6. HPCToolkit
7. Final

Performance Tools: `gprof`



- **Call graph execution profiler**
- **Steps** to obtain profiling data:
  1. **Compile** and link your program with profiling enabled: `gcc -pg -o myprog myprog.c`
  2. **Execute** program to generate the profile file `gmon.out`: `./myprog`
  3. Run `gprof` to **analyse** the profile file: `gprof myprog gmon.out`
- Suitable for profiling full application
  - Retrieval of **call graph**
  - Identification of performance relevant routines
- **Limitations**: Execution times for small routines may be unreliable

gprof: Example (1/2)



Example code:

```c
#include <stdio.h>
#include <stdlib.h>

void init(double *a, int n) {
    for (int i = 0; i < n; i++)
        a[i] = (double)random() / (double)((1ULL << 31) - 1);
}

double dot_product(double *a, double *b, int n) {
    double x = 0;
    for (int i = 0; i < n; i++) x += a[i] * b[i];
    return x;
}

int main() {
    const int n  = 10000000;
    double    *a = (double *) calloc(n, sizeof(double));
    double    *b = (double *) calloc(n, sizeof(double));

    init(a, n); init(b, n);

    printf("dot product = %e\n", dot_product(a, b, n));

    return 0;
}
```

`gprof`: Example (2/2)



`gprof` output:

```
Flat profile:

Each sample counts as 0.01 seconds.
  %   cumulative   self              self     total
 time   seconds   seconds    calls  ms/call  ms/call  name
 69.23      0.09     0.09        2    45.00    45.00  init
 30.77      0.13     0.04        1    40.00    40.00  dot_product

...

                        Call graph (explanation follows)


granularity: each sample hit covers 4 byte(s) for 7.69% of 0.13 seconds

index % time    self  children    called     name
                                                 <spontaneous>
[1]     100.0    0.00    0.13                 main [1]
                 0.09    0.00       2/2           init [2]
                 0.04    0.00       1/1           dot_product [3]
-----------------------------------------------
                 0.09    0.00       2/2           main [1]
[2]      69.2    0.09    0.00       2         init [2]
-----------------------------------------------
                 0.04    0.00       1/1           main [1]
[3]      30.8    0.04    0.00       1         dot_product [3]
-----------------------------------------------
```

Content



1. Time Measurements
2. gprof
3. **PAPI**
4. perf
5. Score-P / Scalasca
6. HPCToolkit
7. Final

Performance Tools: PAPI



PAPI = Performance Application Programming Interface

- Developed at University of Tennessee
- De facto standard to interface with PMU on various architectures with different ISA
- Selected counters supported by PAPI:

`PAPI_TOT_CYC`	Total cycles
`PAPI_REF_CYC`	Reference cycles
`PAPI_TOT_INS`	Instructions completed
`PAPI_FP_INS`	Floating point instructions
`PAPI_L1_DCM`	Level 1 data cache misses
`PAPI_L2_DCM`	Level 2 data cache misses
`PAPI_L3_DCM`	Level 3 data cache misses
`PAPI_BR_INS`	Branch instructions
`PAPI_BR_MPS`	Conditional branch instructions mispredicted

PAPI: Tools and Application Execution



- Selected PAPI tools:
  - `papi_avail`: List available preset events
  - `papi_native_avail`: List natively available events
- Compile application using PAPI: `cc -o myprog.x myprog.c -lpapi`
- Environment variables steering execution:
  - `PAPI_EVENTS`: List of PAPI events
    - Example: `export PAPI_EVENTS="PAPI_TOT_INS,PAPI_TOT_CYC"`
  - `PAPI_OUTPUT_DIRECTORY`: Directory for storing the output

PAPI High-Level API Example (1/2)



Example code:

```c
#include <stdlib.h>
#include <papi.h>

int main()
{
  // Start counting events
  if (PAPI_hl_region_begin("my_kernel") != PAPI_OK)
    exit(1);

  // Some kernel is called here ...

  // Read, stop and reset counters
  if (PAPI_hl_region_end("my_kernel") != PAPI_OK)
    exit(1);

  return 0;
}
```

PAPI Example (2/2)



`PAPI` output:

```
{
  "papi_version":"7.1.0.0",
  "cpu_info":"Hisilicon Kunpeng",
  "max_cpu_rate_mhz":"2600",
  "min_cpu_rate_mhz":"200",
  "event_definitions":{
    "PAPI_TOT_INS":{
      "component":"perf_event",
      "type":"delta"
    },
    "PAPI_TOT_CYC":{
      "component":"perf_event",
      "type":"delta"
    }
  },
  "threads":{
    "0":{
      "regions":{
        "0":{
          "name":"my_kernel",
          "parent_region_id":"-1",
          "cycles":"597",
          "real_time_nsec":"3580",
          "PAPI_TOT_INS":"3847",
          "PAPI_TOT_CYC":"10204"
        }
      }
    }
  }
}
```

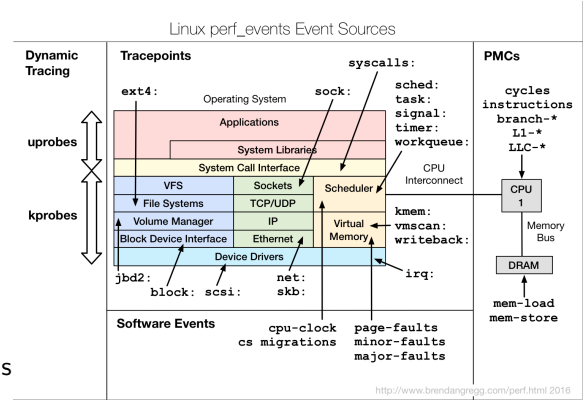
Content



1. Time Measurements
2. gprof
3. PAPI
4. **perf**
5. Score-P / Scalasca
6. HPCToolkit
7. Final

Performance Tools: `perf`

- `perf` is a collection of tools for Linux performance analysis
- Selected tools
  - **stat**: Run a command and gather performance counter statistics
  - **record**: Run a command and record its profile
  - **list**: List all symbolic event types
  - **mem**: Profile memory accesses
  - **ftrace**: Wrapper for the Linux kernel's function tracer (requires privileges)



[B. Gregg]

perf stat



- `perf stat` is a tool for **collecting performance counter statistics**
- **General syntax**:

  `perf stat [-e <EVENT>] <command>`
- Use `perf list` to obtain a list of the symbolic event types
  - Note: Event names may differ on different systems

`perf stat`: Example (1/2)



Example code:

```c
#include <stdio.h>
#include <stdlib.h>

void init(double *a, int n) {
    for (int i = 0; i < n; i++)
        a[i] = (double)random() / (double)((1ULL << 31) - 1);
}

double dot_product(double *a, double *b, int n) {
    double x = 0;
    for (int i = 0; i < n; i++) x += a[i] * b[i];
    return x;
}

int main() {
    const int n  = 10000000;
    double    *a = (double *) calloc(n, sizeof(double));
    double    *b = (double *) calloc(n, sizeof(double));

    init(a, n); init(b, n);

    printf("dot product = %e\n", dot_product(a, b, n));

    return 0;
}
```

`perf stat`: Example (2/2)



`perf` output:

```
 1
 2  Performance counter stats for './myprog.x':
 3
 4          1,649.14 msec task-clock                       #    0.998 CPUs utilized
 5                13      context-switches                 #    7.883 /sec
 6                 1      cpu-migrations                   #    0.606 /sec
 7            39,128      page-faults                      #   23.726 K/sec
 8     1,385,716,301      cycles                           #    0.840 GHz
 9     1,518,719,409      instructions                     #    1.10  insn per cycle
10       329,338,101      branches                         #  199.703 M/sec
11           157,509      branch-misses                    #    0.05% of all branches
12                        TopdownL1                 #     60.8 %  tma_backend_bound
13                                                  #      4.0 %  tma_bad_speculation
14                                                  #      5.4 %  tma_frontend_bound
15                                                  #     29.8 %  tma_retiring
16
17        1.651736403 seconds time elapsed
18
19        1.445731000 seconds user
20        0.205245000 seconds sys
```

perf record



- `perf record` allows to **collect profiling data**
- **General syntax**:

  `perf record [-e <EVENT>] [options] <command>`
- By default, the tool will generate a file `perf.data` that can be inspected using `perf report`

`perf record`: Example (1/2)

Example code compiled with `gcc-14 -O1 -o myprog.x myprog.c`:

```c
#include <stdio.h>
#include <stdlib.h>

void init(double *a, int n) {
    for (int i = 0; i < n; i++)
        a[i] = (double)random() / (double)((1ULL << 31) - 1);
}

double dot_product(double *a, double *b, int n) {
    double x = 0;
    for (int i = 0; i < n; i++) x += a[i] * b[i];
    return x;
}

int main() {
    const int n  = 10000000;
    double    *a = (double *) calloc(n, sizeof(double));
    double    *b = (double *) calloc(n, sizeof(double));

    init(a, n); init(b, n);

    printf("dot product = %e\n", dot_product(a, b, n));

    return 0;
}
```

`perf record`: Example (2/2)



Report generated using `perf record ./myprog.x`:

```
# To display the perf.data header info, please use --header/--header-only options.
#
#
# Total Lost Samples: 0
#
# Samples: 1K of event 'cycles:P'
# Event count (approx.): 1374661176
#
# Overhead  Command   Shared Object      Symbol
# ........  ........  .................  ......................
#
    68.65%  myprog.x  libc.so.6          [.] __random
     9.26%  myprog.x  libc.so.6          [.] __random_r
     6.18%  myprog.x  myprog.x           [.] init
     3.03%  myprog.x  myprog.x           [.] dot_product
     1.16%  myprog.x  myprog.x           [.] random@plt
     0.91%  myprog.x  [unknown]          [k] 0xffffffffaf043aee
     0.91%  myprog.x  [unknown]          [k] 0xffffffffaf20128c
```

Content

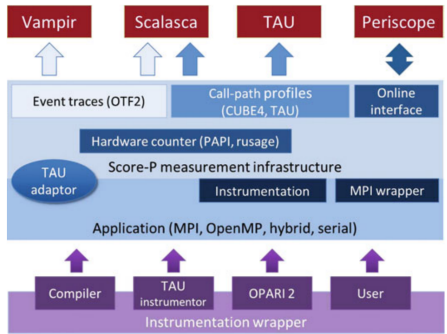


1. Time Measurements
2. gprof
3. PAPI
4. perf
5. **Score-P / Scalasca**
6. HPCToolkit
7. Final

Performance Tools: Score-P

- Score-P is a performance measurement infrastructure for profiling, event tracing and online analysis of (parallel) HPC applications
- Used for different performance analysis tools
- Score-P intercepts the compilation process for automatized code instrumentation

[Knüpfer et al.; 2012]



Scalasca



[Geimer et al.; 2010]

- Collection of trace-based performance analysis tools that have been specifically designed for use on large-scale systems
- Optimised for analyses of applications using MPI, OpenMP, POSIX threads, or hybrid parallelization
- Performance analysis workflow:
  1. Instrument application code
  2. Collect traces by executing the instrumented application
  3. Analyse traces

Scalasca Example (1/3)



Example code:

```c
#include <stdio.h>
#include <stdlib.h>

void init(double *a, int n) {
    for (int i = 0; i < n; i++)
        a[i] = (double)random() / (double)((1ULL << 31) - 1);
}

double dot_product(double *a, double *b, int n) {
    double x = 0;
    for (int i = 0; i < n; i++) x += a[i] * b[i];
    return x;
}

int main() {
    const int n  = 10000000;
    double    *a = (double *) calloc(n, sizeof(double));
    double    *b = (double *) calloc(n, sizeof(double));

    init(a, n); init(b, n);

    printf("dot product = %e\n", dot_product(a, b, n));

    return 0;
}
```

Scalasca Example (2/3): Compile and Run



- Compile with instrumentation:

```
skin gcc-14 -O1 -o myprog.x myprog.c

-> /home/pleiter/opt/scorep-8.4/bin/scorep gcc-14 -O1 -o myprog.x myprog.c
```

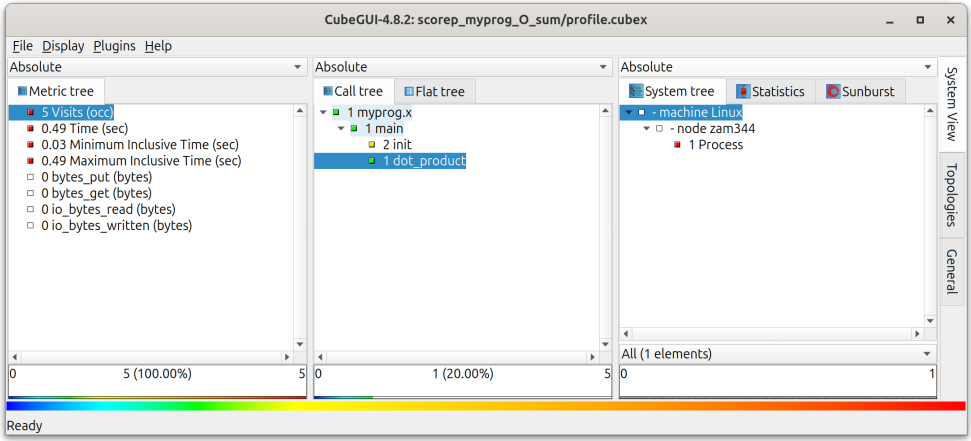
- Run to collect traces:

```
% scan ./myprog.x

S=C=A=N: Scalasca 2.6.2 runtime summarization
S=C=A=N: ./scorep_myprog_O_sum experiment archive
S=C=A=N: Sun Aug 31 13:11:10 2025: Collect start
 ./myprog.x
dot product = 2.499482e+06
S=C=A=N: Sun Aug 31 13:11:13 2025: Collect done (status=0) 3s
S=C=A=N: ./scorep_myprog_O_sum complete.
```

- Use `square` or `cube` to view cube file
  - Here: `scorep_myprog_O_sum/profile.cubex`

Scalasca Example (3/3): Cube View



Score-P / Scalasca Installation



- Score-P and Scalasa are not available through Linux distributions
- Own device
  - Perform build based of Score-P (see `https://www.vi-hps.org/projects/score-p/`)
  - Perform build based of Scalasca (see `https://www.scalasca.org`)
- HAICGU cluster
  - Use pre-compiled Score-P and Scalasca by adding the following directories to your `PATH`
    - `/home/dpleiter/opt/scorep-9.2/bin`
    - `/home/dpleiter/opt/scalasca-2.6.2/bin`
  - Use Cube on your own device

Content



1. Time Measurements
2. gprof
3. PAPI
4. perf
5. Score-P / Scalasca
6. **HPCToolkit**
7. Final

Performance Tools: HPCToolkit



[Adhianto et al.; 2010]

- HPCToolkit uses **statistical sampling** to collect performance data
- **Key capabilities**:
  - **Measure** and collect performance data
  - **Application analysis**: recover data structure from application binary
  - **Performance attribution**
  - **Presentation** of performance metrics and associated source code
- **Challenge**: HPCToolkit has become difficult to install

HPCToolkit Workflow



1. **Measurement**: `hpcrun myprog.x`
2. **Recover** static source code structure: `hpcstruct myprog.x`
3. **Correlate** call path or flat metrics with static source code structure: `hpcprof`

Content



1. Time Measurements
2. gprof
3. PAPI
4. perf
5. Score-P / Scalasca
6. HPCToolkit
7. **Final**

Further Reading



- PAPI
  - https://github.com/icl-utk-edu/papi/wiki/
- perf
  - https://perfwiki.github.io
- Scalasca
  - https://www.scalasca.org/scalasca/software/scalasca-2.x/documentation.html

Questions?



[Wiki Media]